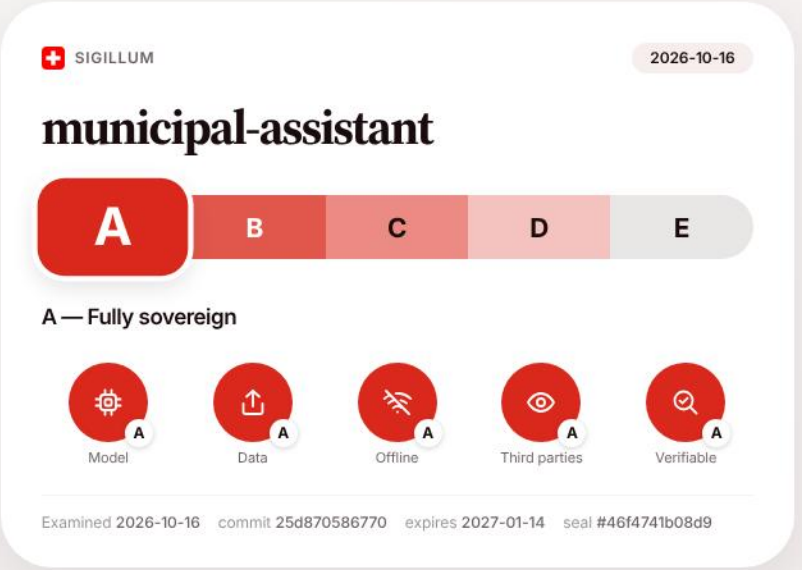
Sigillum

Sovereign? Prove it.

A sovereignty seal for AI tools — from A to E, with proof behind every answer. Built with Apertus, applied to ourselves first.

Sigillum is Latin for “seal”, as Apertus is Latin for “open”.



THE PROBLEM

Everyone says “sovereign”. Nobody measures it.

Unverifiable promises

A public buyer cannot tell a sovereign tool from one that only says so.

Hidden dependencies

An app on an open model can still send data to a foreign API, an analytics service or a CDN — or stop when the connection drops.

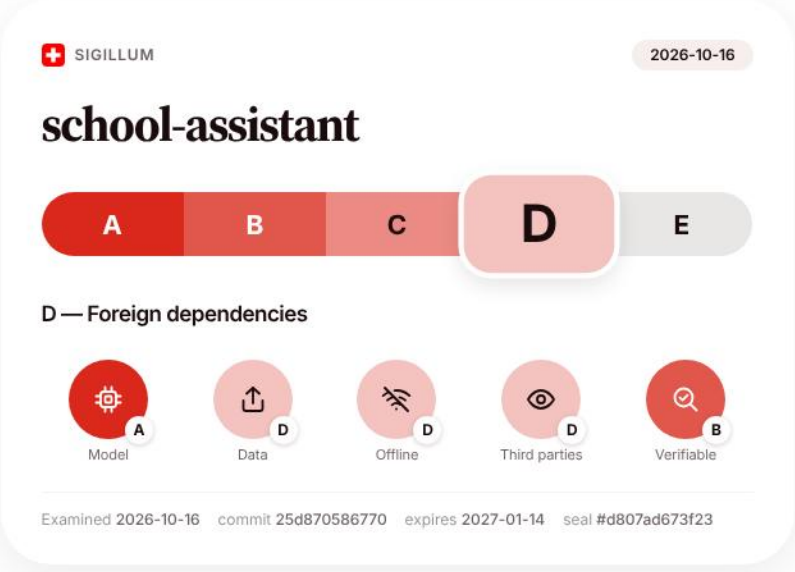
No common yardstick

Two teams that say “sovereign” do not mean the same thing. Organisers cannot tell which projects to showcase.

THE IDEA

One letter. Five pictograms. A proof for each.

Before the energy label, we bought fridges on the seller's word. Sigillum does for AI sovereignty what the label did for energy: a simple sign, backed by a public method.



THE FIVE QUESTIONS

Five questions. One proof for each.

Where does the model live?

- A — On hardware the organisation controls
- E — Closed model at a foreign company
- ? — unknown — never guessed

Where does the data go?

- A — Nothing leaves
- E — Questions and documents leave the country
- ? — unknown — never guessed

What happens without internet?

- A — Everything works — and it is tested
- E — Everything stops
- ? — unknown — never guessed

Who else is in the loop?

- A — Nobody: no tracking, no telemetry
- E — Trackers observe the users
- ? — unknown — never guessed

Can it be verified?

- A — Licence, reproducible build, tests
- E — Nothing to check
- ? — unknown — never guessed

Sigillum reads code and configuration — never user data.

THE METHOD

Three rules make the grade credible.

1

The proof rule

A claim without a trace never earns more than C. Missing information is shown as **unknown** — and caps the grade at C.

2

The weakest link

The final grade is the worst of the five answers. Perfect on four but data goes abroad? Not an A.

3

A dated, tamper-evident seal

Date, commit, method version and a SHA-256 fingerprint. It expires after 90 days: a project that changes must be examined again.

Apertus v1.5 8B · local

Apertus reads, reconciles and explains. The evidence decides.

The grade never depends on the model. Apertus does what fixed rules cannot — and every answer is checked before use.

1 **Reads**
Extracts what the documentation claims about sovereignty. A quote must exist word for word.

2 **Reconciles**
Sets each claim against the code traces and names the gap. It must cite evidence IDs that exist.

3 **Explains**
Writes — or confirms — the verdict in plain German, French, Italian and Romansh. A line in the wrong language is rejected.

Per project: up to 7 Apertus calls (read 1 · reconcile 1 · explain 5) · about 5 minutes on 4 CPU cores, no GPU · every rejected answer is logged as feedback for the Apertus team

THE MOMENT THAT MATTERS

When what is said and what is done diverge, Sigillum shows it.

THE PROJECT SAYS

“Everything runs locally: no data leaves the school.”

README.md

THE TRACES SHOW

Traces show data being sent to external services outside the school.

GLOBAL cdn.jsdelivr.net · app.py:7

US data.streamlit.io · app.py:1

Found by Apertus E04 E07 — every ID points to a file and a line

Calmly, with evidence, without accusing anyone. Example project built by the Sigillum team.

FOUR NATIONAL LANGUAGES

Explained in the four national languages.

So that sovereignty is understood by the people it is meant to protect. Here: "Where does the data go?" for the closed-model example.

DEUTSCH

Ihre Fragen und Dokumente verlassen die Schweiz und werden von OpenAI in den Vereinigten Staaten verarbeitet.

Written by Apertus

FRANÇAIS

Vos questions et vos documents quittent la Suisse, hébergés par OpenAI (États-Unis).

Written by Apertus

ITALIANO

Le vostre domande e i vostri documenti lasciano la Svizzera.

Written by Apertus

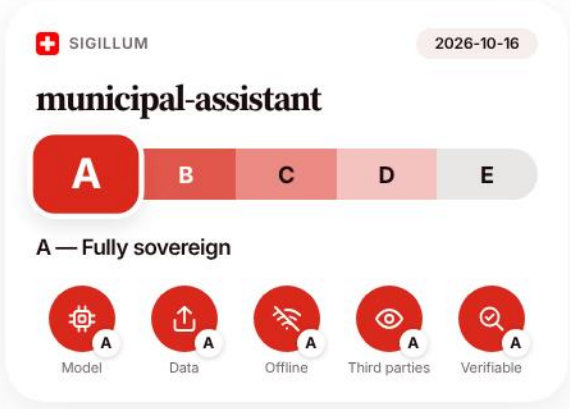
RUMANTSCH

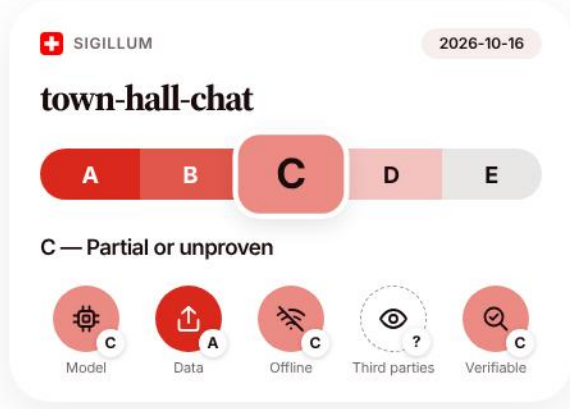
Vossas dumondas e Voss documents bandunan la Svizra.

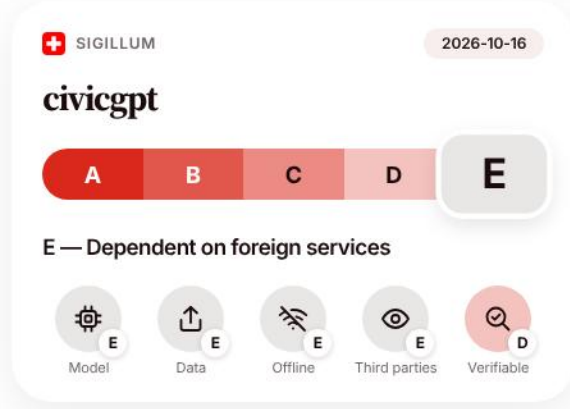
Written by Apertus

SCENE 1 · A MUNICIPAL CIO

Three vendors say “sovereign”. Two minutes to decide.







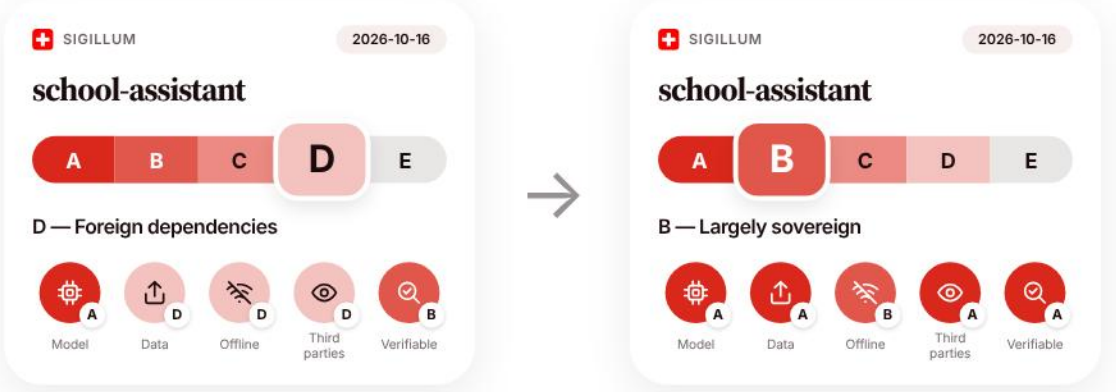
An A; a C with “unknown” on question 4 — the question to ask that vendor; an E, because the data leaves the country.

SCENE 2 · A TEAM THAT WANTS TO IMPROVE

Not a sanction: a roadmap.

Each report lists concrete fixes. The team fixed three of them and moved from D to B. The history keeps the before and after.

1. Remove the CDN script — bundle it
2. Switch off Streamlit usage statistics
3. Serve Apertus on the school's server (llama.cpp)

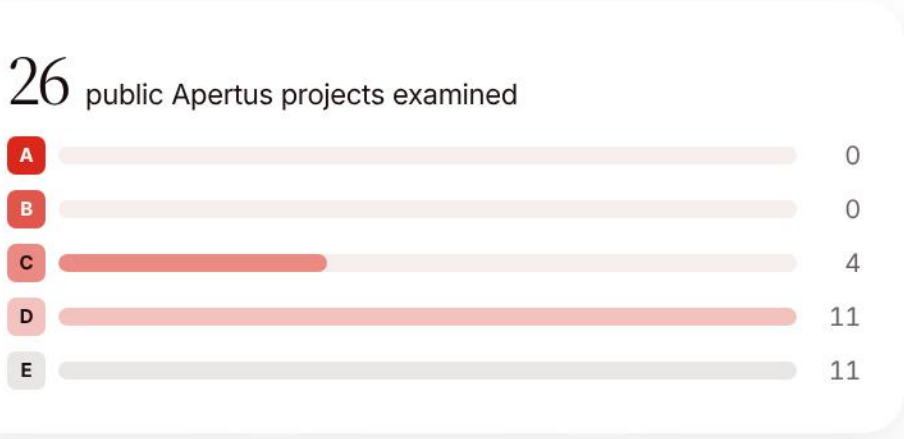


THE HEADLINE · AGGREGATED AND ANONYMOUS

We measured the public Apertus ecosystem.

Hack Apertus submissions are public only after 24 October. So we applied the seal to every public Hugging Face Space that uses Apertus — without naming anyone.

Static analysis of public code only · no request to running apps · no data about users · the aggregation code cannot output names.



42% call the model through a foreign service

73% ship telemetry on by default

0% proven or likely to work offline

Most frequent foreign dependency: Gradio usage analytics (17/26) — a one-line fix.

SCENE BONUS · THE JURY OPENS OUR SEAL

We started with ourselves.

Sigillum's first examination of itself gave **D**: no licence file in the project folder. Fixed, re-examined: **A**. Then we published our limits.

- Static analysis only — runtime-only behaviour can be missed
- Hand-curated registry: unknown hosts are reported as unknown, never as safe
- Apertus 8B makes occasional German errors; Romansh needs a native proof-reader
- Exclusions are self-declared — and published in every report



SOVEREIGN DEPLOYABILITY · AIR-GAPPED

The tool runs offline — and proves it.



0

outbound requests at runtime (except the configured Apertus endpoint)

18 + 5

tests and sample audits passing inside `docker --network none`

70 ms

deterministic examination per project · Apertus ≈ 5 min on 4 CPU cores · CHF 0 marginal cost

TRACK 2B CRITERIA

Built for the jury's five criteria.

Purposeful use of AI	Apertus reads, reconciles and explains — where fixed rules fail; the evidence sets the grade
Technical rigour	Proof rule, weakest link, “unknown” shown, answers checked and rejections counted, 18 tests, false positives fixed and documented
Value, cost & scalability	70 ms per project deterministic, minutes with Apertus on a CPU; any repository; one shared language for buyers, teams and organisers
Sovereign deployability	It is the subject: air-gapped by design, offline test in Docker, Sigillum examined by Sigillum
Implementation feasibility	Working end to end today: `make run`, CLI, web UI, JSON reports, badges

Sovereignty is not declared. It is proven.

So that sovereign AI does not remain a slogan, Sigillum makes it a common, open and verifiable rule of the game — ready for the on-site hacks and the May 2027 finals.

Apertus is open so that trust can be verified. Sigillum makes it readable.

Open source · Apache-2.0 · CC-BY-4.0 · Transparency instrument, not an official certification